

# Project 4 — C Subset Compiler for LLVM IR

## Requirements

| Tool       | Version                                        |
|------------|------------------------------------------------|
| Java (JDK) | 8 or later                                     |
| ANTLR4     | 4.13.2 (included as antlr-4.13.2-complete.jar) |
| clang      | Any modern version                             |

## How to Compile and Run

### Step 1 — Build the compiler

```
make build
```

Runs ANTLR4 to generate the lexer/parser from `myCompiler.g4`, then compiles all `.java` files.

### Step 2 — Compile a C source file to LLVM IR

```
java -cp ./antlr-4.13.2-complete.jar myCompiler_test <input>.c > <output>.ll
```

### Step 3 — Link and execute

```
clang <output>.ll myRuntime.c -o <program> -lm  
./<program>
```

## Makefile shortcuts

```
make run TEST=test      # build, compile test.c → test.ll, link, and run  
make gen-ll TEST=test   # compile test.c → test.ll only (no link/run)  
make run-all           # build, compile, link, and run all test files  
make gen-ll-all         # compile all test files to .ll only (no link/run)
```

## Supported C Subset

### Types

| Type   | LLVM   | Notes |
|--------|--------|-------|
| int    | i32    |       |
| float  | float  |       |
| double | double |       |

| Type                          | LLVM         | Notes                               |
|-------------------------------|--------------|-------------------------------------|
| void                          | void         | return type only                    |
| char                          | i8           |                                     |
| bool                          | i1           | true/false literals supported       |
| long long, unsigned long long | i64          |                                     |
| unsigned, long, short, etc.   | i32          | mapped to int                       |
| Pointer T*                    | ptr          | any depth                           |
| Array T[N], T[N][M]           | [N x T]      | stack-allocated                     |
| struct, union                 | %struct.Name | named fields, anonymous blocks      |
| typedef                       | —            | scalar and function-pointer aliases |

## Statements

| Statement                                               | Notes                                                                  |
|---------------------------------------------------------|------------------------------------------------------------------------|
| Assignment =, +=, -=, *=, /=, %=, &=, \ =, ^=, <<=, >>= | scalars, array elements, struct fields, pointer-to-struct fields, *ptr |
| ++ / --                                                 | prefix and postfix forms                                               |
| if / if-else                                            | arbitrary nesting, bare body or braced                                 |
| while                                                   | break and continue supported                                           |
| for                                                     | C99 init declaration, multiple update expressions, break / continue    |
| do-while                                                | break and continue supported                                           |
| switch / case / default                                 | integer/char/enum case values, C fall-through, break                   |
| return                                                  | coerces to declared return type                                        |
| break / continue                                        | target nearest enclosing loop or switch                                |
| goto / label                                            | forward and backward jumps within a function                           |
| Function call                                           | value-returning (in expression) or void (as statement)                 |

## Expressions

| Feature                     | Notes                                  |
|-----------------------------|----------------------------------------|
| Arithmetic +, -, *, /, %    | int, float, double; implicit promotion |
| Bitwise &, \ , ^, ~, <<, >> | integer operands only                  |

| Feature                                | Notes                                                |
|----------------------------------------|------------------------------------------------------|
| Logical &&, \ \ , !                    | short-circuit emitted as <code>and i1 / or i1</code> |
| Comparison >, >=, <, <=, ==, !=        | all scalar types                                     |
| Ternary ? :                            | result type is wider branch                          |
| Comma operator (a, b, c)               | inside parentheses only                              |
| Unary -, +                             | negation / no-op                                     |
| Address-of &id                         | yields stack-slot address                            |
| Dereference *expr                      | loads through pointer                                |
| ## operator                            | <code>a ## b</code> = $a^b + b^a$ (float result)     |
| Type cast (T)                          | scalar casts and pointer casts                       |
| sizeof(T)                              | compile-time constant                                |
| sizeof(struct S)                       | sum of field sizes                                   |
| Array element a[i], a[i][j]            | read and write                                       |
| Struct member .field, ->field          | read and write                                       |
| Function-pointer call arr[i](args)     | via pointer array element                            |
| NULL, true, false                      | integer / bool literals                              |
| Hex literals 0xFF                      | <code>int</code>                                     |
| Char literals 'A', '\n'                | <code>int</code>                                     |
| Float suffix 1.5f                      | <code>float</code> literal                           |
| String literal adjacent concat "a" "b" | produces single string global                        |

## Top-Level Declarations

| Feature                     | Notes                                                                                   |
|-----------------------------|-----------------------------------------------------------------------------------------|
| Global variables            | scalar and array; zero-initialised in IR                                                |
| enum                        | hex values, expression values, optional implicit counter                                |
| struct                      | named fields, pointer fields, bit-fields (width ignored), anonymous union/struct blocks |
| union                       | treated as struct                                                                       |
| typedef                     | scalar alias, function-pointer alias                                                    |
| Function prototype          | for mutual recursion                                                                    |
| #include, #define, #if, ... | silently skipped by lexer                                                               |

# Code Optimizations

All optimisations happen during code generation with no separate pass.

## 1. Constant Folding

Arithmetic, bitwise, comparison, and conversion operations on compile-time constants are evaluated at compile time — no instruction is emitted.

```
a = 2 + 3;           // store i32 5
x = 1.5 * 2.0;       // store double <hex for 3.0>
y = 2.0 ## 3.0;      // store float <hex for 17.0>
z = 0xFF & 0x0F;     // store i32 15
```

## 2. Algebraic Identity Simplifications

Trivially redundant operations are eliminated:

| Expression     | Result |
|----------------|--------|
| $x + 0, x - 0$ | $x$    |
| $x * 1, x / 1$ | $x$    |
| $x * 0$        | $0$    |

## 3. Constant Propagation

A variable assigned a constant is tracked; subsequent reads use the constant directly (no load emitted). The table is conservatively cleared at every control-flow merge point and at `scanf` calls.

```
a = 10;           // constSlot[a] = 10
b = a + 5;        // → b = 15, no load
c = b * 2;        // → c = 30, no load
```

## 4. Constant Type Conversion

Coercions between `int`, `float`, and `double` constants are resolved at compile time:

```
float x = 5;      // store float 0x4014000000000000 (no sitofp instruction)
int  n = 2.9;     // store i32 2 (no fptosi instruction)
```

---

## Test Programs

### test.c — Comprehensive stress test

Exercises all basic and extended features together: multiple user-defined functions, `scanf/print`, implicit and explicit type conversion, `##` operator, nested `if/else`, `while` loop, compound operators.

**Run:** `make run TEST=test`

---

## **test\_types.c — Extended type features**

Hex literals, char literals, NULL/true/false, double arithmetic, sizeof, ternary operator, bitwise compound assignments (&=, |=, ^=, <<=, >>=), goto, global variables.

**Run:** make run TEST=test\_types

---

## **test\_control.c — Control flow and operators**

for loop, for with continue, do-while, && and ||, logical !, bitwise operators (&, |, ^, <<, >>), bitwise NOT ~.

**Run:** make run TEST=test\_control

---

## **test\_switch.c — Switch statement**

Basic integer switch, switch on enum value, case fall-through via absent break, break inside while loop, switch with no default.

**Run:** make run TEST=test\_switch

---

## **test\_new.c — Arrays and structs**

enum with explicit and implicit values, 1-D array, 2-D array, struct with int fields, struct with float fields, function taking an array, member reads/writes.

**Run:** make run TEST=test\_new

---

## **test\_extra.c — Typedefs, global arrays, compound ops**

typedef for int and float, enum with hex initialisers, adjacent string literal concatenation, global array read/write, compound assignment on array elements and struct fields, double constant folding.

**Run:** make run TEST=test\_extra

---

## **test\_str.c — Char arrays and strings**

char array declaration, string-literal assignment via strcpy, scanf %s into a char array, printf %s.

**Run:** make run TEST=test\_str

---

## testAll.c — Full-feature integration test

Combines `#include` directives (skipped), `#define` macros (skipped), complex `struct/union/typedef/enum` declarations, function prototypes for mutual recursion, global variables, pointer arithmetic, designated initialisers, anonymous union/struct, bit-fields, `for` with inner declarations, `do-while`, `switch` fall-through, `goto`, function pointer array, `malloc/strcpy/free`, and mutually recursive functions `isEven/isOdd`.

**Run:** `make run TEST=testAll`

---

## Runtime Library (myRuntime.c)

Implements the `##` operator and exposes standard stream symbols:

```
float __hashOp(float a, float b) {  
    return powf(a, b) + powf(b, a);  
}  
  
FILE** _get_stdout_ptr(void) { return &stdout; }  
FILE** _get_stderr_ptr(void) { return &stderr; }  
FILE** _get_stdin_ptr(void) { return &stdin; }
```

`__hashOp` is declared in the generated IR as `declare float @__hashOp(float, float)` and linked at the `clang` step.

---

## File Structure

|                                               |                                                                     |
|-----------------------------------------------|---------------------------------------------------------------------|
| <code>myCompiler.g4</code>                    | – ANTLR4 grammar (lexer + parser + code generator)                  |
| <code>myCompiler_test.java</code>             | – Driver: reads <code>.c</code> file, runs parser, prints LLVM IR   |
| <code>myRuntime.c</code>                      | – Runtime library ( <code>__hashOp</code> + stream pointer helpers) |
| <code>Makefile</code>                         | – Build and run targets                                             |
| <code>antlr-4.13.2-complete.jar</code>        | – ANTLR4 runtime (required)                                         |
| <code>test.c / test.ll</code>                 | – Comprehensive stress test                                         |
| <code>test_types.c / test_types.ll</code>     | – Extended type features                                            |
| <code>test_control.c / test_control.ll</code> | – Control flow and operators                                        |
| <code>test_switch.c / test_switch.ll</code>   | – Switch/case with enum values and fall-through                     |
| <code>test_new.c / test_new.ll</code>         | – Arrays, multi-dim arrays, structs                                 |
| <code>test_extra.c / test_extra.ll</code>     | – Typedefs, global arrays, compound ops                             |
| <code>test_str.c / test_str.ll</code>         | – Char arrays and string operations                                 |
| <code>testAll.c / testAll.ll</code>           | – Full-feature integration test                                     |